

EN.685.652.81.SP25 Data Engineering Principles and Practice

Final Project

Group 4 Members:

Zinia Mahabub

Maria Jose Hito Ramos

Tori-Ann Holness

Tonya Sosa

Due: April 22, 2025

Project Description	2
Datasets	2
Data Transformation Process.....	2
Database Schema.....	13
Automation	14
API	14
Instructions to Run the Pipeline.....	16

Project Description

The general concept of this project is to find the correlation between the weather and covid-19 cases as well as the correlation between covid-19 surge and unemployment rates. All the data has been derived from a randomly chosen year: 2021. The project should allow users to explore whether there was any impact of weather on covid-19 cases and if both the weather and covid-19 had any impact on the unemployment rates. The ETL pipeline starts with transforming the data, then merging and loading the data to a PostgreSQL database, setting up a DAG so that users can automate the process and trigger new data loads into the database, and an API to allow users to query insights from the clean data.

Datasets

The first dataset is covid-19 data from [The Center for Systems Science and Engineering \(CSSE\) at JHU](#). It is a csv file that contains daily covid cases by state and county from 1/22/2020 to 3/9/2023.

The second dataset is unemployment rate data from [Kaggle](#). It is a csv file that contains the employment and unemployment rate for every state for the year 2021. The data is from 1/1/2021 to 12/1/2021.

The third dataset is weather data obtained from the [Visual Crossing API](#). It was used to generate JSON files using API key for each desired state for the year 2021.

Each dataset was cleaned and filtered to 10 selected U.S. states: California, Connecticut, Florida, Georgia, Maryland, Michigan, New York, Pennsylvania, Texas, and Washington.

Data Transformation Process

- Covid-19 Data: The first step in the data transformation process was to load the data into a python data frame. Unused columns were dropped for optimizing performance and the resulting columns were state, county, state& county, and the individual dates across the

columns. The file had the daily dates across the columns and to aggregate total covid cases, the date columns were unpivoted and the result was one date column with a values column for the number of cases. Cases were then summed up by 'state' and filtered to the 10 desired states mentioned above. The month/year was then extracted and filtered to 2021. Lastly, the data was aggregated by month/year and state. The resulting data frame was 3 columns: state, month/year, and number of cases and saved under *cleaned_covid.xlsx*.

- Unemployment Data: The notebook that cleans the data executes a structured data transformation. Initially, the raw CSV is ingested using pandas, followed by standardization of column headers to ensure compatibility with database schemas. Key numeric fields—namely *civilian_population*, *labor_force*, *employment*, and *unemployment*—are parsed from string format by removing separators and casting to integer type. Temporal granularity is enhanced by synthesizing a *month/year* identifier in MM/YYYY format, after which the original *year* and *month* columns are dropped to eliminate redundancy. The dataset is then subset to include only entries from the year 2021 and a predefined list of ten U.S. states. State name strings are normalized via whitespace trimming and title-casing, and FIPS codes are reformatted to preserve leading zeros. Derived features include the *employment-to-population* ratio and *unemployment rate*, computed via element-wise division. The final stage involves deduplication and null handling (dropping records with missing *state* or *month/year* values), culminating in the export of the cleaned dataset to a CSV file: *cleaned_unemployment_data.csv*.

- Weather Data:

The weather data was obtained from Visual Crossing Weather API, a comprehensive weather data service that provides historical, current, and forecast weather information through its API. To ensure geographic diversity and alignment with COVID and unemployment datasets, weather data were retrieved for 10 U.S. states randomly chosen by the group members. Each API call was structured to retrieve daily observations for each state across 2021. Fields such as *tempmax*, *tempmin*, *temp*, *feelslike*, *humidity*, *precip*, *preciptype*, *conditions*, and *description* were included.

To ensure security and modularity, a *config.json* file was created in the project directory with the following content:

```
{ } config.json > ...
1   {
2   |   "VISUAL_CROSSING_API_KEY": "your_api_key_here"
3   }
```

The users will need to enter their own uniquely generated API keys retrieved from the visual crossing website by creating an account.

Example of the API endpoint format:

https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/timeline/{state}/2021-01-01/2021-12-31?unitGroup=us&key=YOUR_API_KEY&contentType=json

weather_data_retrievedfromAPI.py:

```
weather_data_retrievedfromAPI.py ×
python_scripts > weather_data_retrievedfromAPI.py > ...
1  import requests
2  import json
3  from pathlib import Path
4
5  # Load API key from config.json
6  config_path = Path("config.json")
7  with config_path.open() as config_file:
8      config = json.load(config_file)
9
10 api_key = config.get("VISUAL_CROSSING_API_KEY")
11
12 # List of states to retrieve weather data for
13 states = [
14     "California", "Connecticut", "Florida", "Georgia", "Maryland",
15     "Michigan", "New York", "Pennsylvania", "Texas", "Washington"
16 ]
17
18 # Define API endpoint and base params
19 base_url = "https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/timeline"
20
21 # Set output folder
22 output_folder = Path("data/weather_data_json")
23
24 output_folder.mkdir(parents=True, exist_ok=True)
25
26 # Request and save data for each state
27 for state in states:
28     url = f"{base_url}/{state}/2021-01-01/2021-12-31"
29     params = {
30         "unitGroup": "us",
31         "key": api_key,
32         "contentType": "json"
33     }
```

```
34
35     response = requests.get(url, params=params)
36     if response.status_code == 200:
37         data = response.json()
38         file_path = output_folder / f"weather_data_{state[:2].upper()}.json"
39         with open(file_path, "w") as f:
40             json.dump(data, f, indent=4)
41         print(f"Saved data for {state} to {file_path}")
42     else:
43         print(f"Failed to retrieve data for {state}: {response.status_code}")
```

For user-friendliness, there is a .ipynb file also added where it shows how to get the json files individually for each state. It was not possible to retrieve the data for all states to be on one json file from this specific website. So, each call was made individually for each state.

Each response then was saved locally as a JSON file, such as *weather_data_TX.json*. The cleaning and aggregation were done in the *clean_data_sql_merge.py* Python script. Each JSON file was loaded and parsed using Python's json module. Each record from the daily data was stored in a pandas DataFrame with fields such as State, Date (converted to Month/Year or MM/YYYY), and various weather metrics. The data was then grouped by Month/Year and State, aggregated using mean for numeric columns (e.g., temperature, humidity), and mode was used for categorical fields (e.g., conditions). The final cleaned dataset was saved as *cleaned_weather_data.csv*.

- CSV Output: After the data was transformed and merged, it was exported to a csv (*combined_data.csv*) file to use in the API automation process.
- Loading Data to Postgres Database: The 3 datasets were merged by state and month/year columns. Since the relationships between the individual datasets were one-to-one, they were merged by an inner join. The Postgres table is setup to have an id column, and the month/year and description columns are TEXT. All value columns are either in real number format or integer—the state column, fips code, participation type, and condition columns— range from VARCHAR 10 to 100. The VARCHAR limit for the *conditions* column was set to 100 to allow for various length of sentences, most values in this column are a sentence. The VARCHAR limit for the *fips code* was set to 10 since the fips code for *states* is up to 2 characters and in case a longer one came in by user input, it is a 10-character limit. The INTEGER type was chosen for numbers that are not decimals, the REAL type was chosen for columns that have decimal values such as the percentage columns.

Code for data transformation process for each dataset and loading into database:

```
# %%
import pandas as pd
from pathlib import Path
import json

# load in covid data

# path to data folder
covid_file_path = Path("/home/jhu/data/time_series_covid19_confirmed_US.csv")

# load in csv
df = pd.read_csv(covid_file_path)
```

```

# %%
# remove unnecessary columns
df = df.drop(columns=['UID', 'iso2',
'iso3', 'code3', 'FIPS', 'Country_Region', 'Lat', 'Long_'])
# unpivot date columns to get one date column
df_unpivot = df.melt(id_vars=['Admin2', 'Province_State', 'Combined_Key'],
var_name='Date', value_name='Cases')

# group by states and sum the cases per cities in each state
df_states = df_unpivot.groupby(['Date', 'Province_State'], as_index=False)['Cases'].sum()

# Filter data to desired states
df_filtered =
df_states[df_states['Province_State'].isin(['California', 'Connecticut', 'Florida', 'Georgia',
',
Maryland', 'Michigan', 'New
York', 'Pennsylvania',
Texas', 'Washington'])]]

## copy of the df
df_filtered = df_filtered.copy()

# set date as string
df_filtered['Date'] = df_filtered['Date'].astype('string')

# extract before first and after last / for month and year columns
df_filtered['month'] = df_filtered['Date'].str.split('/').str[0]
df_filtered['year'] = df_filtered['Date'].str.split('/').str[-1]

# adding leading 0s to month
df_filtered['month'] = df_filtered['month'].astype(str).str.zfill(2)

# merge month and year column by /
df_filtered['month/year'] = df_filtered['month'] + '/' + df_filtered['year']

# filter data to 2021
df_2021= df_filtered[df_filtered['year'] == '2021']

# aggregate by month
df_clean = df_2021.groupby(['month/year', 'Province_State'],
as_index=False)['Cases'].sum()
df_clean = df_clean.rename(columns={'Province_State': 'state', 'Cases': 'cases'})
df_clean['month/year'] = df_clean['month/year'].astype('string')

output_path_covid = Path("/home/jhu/data/cleaned_covid.xlsx")

# save as csv to data folder

```

```

df_clean.to_excel(output_path_covid, index=False)

# %%
# load in unemployment data

# path to data folder
unemp_file_path = Path("/home/jhu/data/Unemployment in America Per US State.csv")

# load in csv
unem_df = pd.read_csv(unemp_file_path)

# Standardize column names for database compatibility
unem_df.columns = [
    "fips_code", "state", "year", "month", "civilian_population", "labor_force",
    "population_percent", "employment", "employment_percent", "unemployment",
    "unemployment_percent"
]

# Convert numeric columns from string with commas to integers
numeric_cols = ["civilian_population", "labor_force", "employment", "unemployment"]
for col in numeric_cols:
    unem_df[col] = unem_df[col].str.replace(",", "").astype(int)

# Create a single 'month/year' column in MM/YYYY format
unem_df["month/year"] = unem_df.apply(lambda x:
    f"{int(x['month']):02d}/{int(x['year'])}", axis=1)

# Drop the original year and month columns
unem_df.drop(columns=["year", "month"], inplace=True)

# Filter data for the year 2021
unem_df = unem_df[unem_df["month/year"].str.endswith("/2021")]

# Standardize the state column
unem_df["state"] = unem_df["state"].str.strip().str.title()

# Filter for specific states
selected_states = ["California", "Connecticut", "Florida", "Georgia", "Maryland",
    "Michigan", "New York", "Pennsylvania", "Texas", "Washington"]
unem_df = unem_df[unem_df["state"].isin(selected_states)]

# Convert FIPS code to string to retain leading zeros
unem_df["fips_code"] = unem_df["fips_code"].astype(str).str.zfill(2)

# Creating Additional Features

```

```

unem_df["employment_to_population_ratio"] = unem_df["employment"] /
unem_df["civilian_population"]
unem_df["unemployment_rate"] = unem_df["unemployment"] / unem_df["labor_force"]

# Remove duplicates
unem_df.drop_duplicates(inplace=True)

# Handle missing values (Drop rows where state or date is missing)
unem_df.dropna(subset=["state", "month/year"], inplace=True)

output_path_unem = Path("/home/jhu/data/cleaned_unemployment_data.csv")

# save as csv to data folder
unem_df.to_csv(output_path_unem, index=False)

# %%
# load in weather data

# path to data folder
weather_folder_path = Path("/home/jhu/data/weather_data_json")

json_files = list(weather_folder_path.glob("weather_data_*.json"))

all_records = []

# Extract daily records from all JSON files
for file_path in json_files:
    with open(file_path, "r") as f:
        data = json.load(f)
        if isinstance(data, list):
            data = data[0]
        state = data.get("address", "Unknown").strip().title()
        for day in data.get("days", []):
            date = day["datetime"]
            mm_yyyy = f"{date[5:7]}/2021"
            record = {
                "State": state,
                "Month/Year": mm_yyyy,
                "TempMax": day.get("tempmax"),
                "Temp": day.get("temp"),
                "TempMin": day.get("tempmin"),
                "FeelsLike": day.get("feelslike"),
                "Humidity": day.get("humidity"),
                "Precipitation": day.get("precip"),
            }

```



```

        "PrecipitationType": ', '.join(day["preciptype"]) if
isinstance(day.get("preciptype"), list) else day.get("preciptype"),
        "Conditions": day.get("conditions"),
        "Description": day.get("description"),
    }
    all_records.append(record)

# Create DataFrame
w_df = pd.DataFrame(all_records)

# Filter for selected 10 states
selected_states = [
    "California", "Connecticut", "Florida", "Georgia", "Maryland",
    "Michigan", "New York", "Pennsylvania", "Texas", "Washington"
]
w_df = w_df[w_df["State"].isin(selected_states)]

# Group by Month/Year and State – calculate monthly averages
monthly_summary = w_df.groupby(["Month/Year", "State"]).agg({
    "TempMax": "mean",
    "Temp": "mean",
    "TempMin": "mean",
    "FeelsLike": "mean",
    "Humidity": "mean",
    "Precipitation": "mean",
}).reset_index()

# Find most common values for categorical columns
def get_mode(series):
    return series.mode().iloc[0] if not series.mode().empty else None

modes = w_df.groupby(["Month/Year", "State"]).agg({
    "PrecipitationType": get_mode,
    "Conditions": get_mode,
    "Description": get_mode
}).reset_index()

# Merge summary and modes
monthly_summary = monthly_summary.merge(modes, on=["Month/Year", "State"])

# Round all numeric columns to 1 decimal place
numeric_cols = monthly_summary.select_dtypes(include=["float64", "int64"]).columns
monthly_summary[numeric_cols] = monthly_summary[numeric_cols].round(1)

# Reorder columns and save
final_df = monthly_summary[[
    "Month/Year", "State", "TempMax", "Temp", "TempMin", "FeelsLike",
    "Humidity", "Precipitation", "PrecipitationType", "Conditions", "Description"
]]

```

```

]]

# Save to CSV

output_path_w = Path("/home/jhu/data/cleaned_weather_data.csv")

# save as csv to data folder
final_df.to_csv(output_path_w, index=False)

# %%

# %%

unemployment_path = Path("/home/jhu/data/cleaned_unemployment_data.csv")
weather_path = Path("/home/jhu/data/cleaned_weather_data.csv")
covid_path = Path("/home/jhu/data/cleaned_covid.xlsx")

# load in files
df_unemp = pd.read_csv(unemployment_path)
df_weather = pd.read_csv(weather_path)
df_covid = pd.read_excel(covid_path)

# merge covid and unemployment data, merging directly since there is one row per
month/year and state
merged_covid_unemp = pd.merge(df_covid, df_unemp, on=['month/year', 'state'],
how='inner')

# merge result and weather data
final_df = pd.merge(merged_covid_unemp, df_weather, left_on=['month/year', 'state'],
right_on=['Month/Year', 'State'], how='inner')
final_df.rename(columns={'cases': 'covid_cases'}, inplace=True)
final_df.drop(columns=['State', 'Month/Year'], inplace=True)

# making reference tables for total of 5 tables
date_df = final_df[['month/year']].drop_duplicates().reset_index(drop=True)
location_df = final_df[['state', 'fips_code']].drop_duplicates().reset_index(drop=True)

# save to folder
output_path_all = Path("/home/jhu/data/combined_data.csv")

# save as csv to data folder
final_df.to_csv(output_path_all, index=False)

final_df = final_df.astype(object)
final_df = final_df.where(pd.notnull(final_df), None)

```

```

# %%
## copy excel to sql db and print 5 rows

## Establish the connection
import psycopg2
import pandas as pd
conn = psycopg2.connect(
    dbname='final_project',
    user='jhu',
    password='jhu123',
    host='postgres', # Adjust if needed for Docker service name
    port='5432'
)
# create a cursor object
cur = conn.cursor()

#query to make table
create_table = """
    CREATE TABLE project (
        id SERIAL PRIMARY KEY,
        month_year TEXT,
        state VARCHAR(50),
        covid_cases INTEGER,
        fips_code VARCHAR(10),
        civilian_population INTEGER,
        labor_force INTEGER,
        population_percent REAL,
        employment INTEGER,
        employment_percent REAL,
        unemployment INTEGER,
        unemployment_percent REAL,
        employment_to_population_ratio REAL,
        unemployment_rate REAL,
        TempMax REAL,
        Temp REAL,
        TempMin REAL,
        FeelsLike REAL,
        Humidity REAL,
        Precipitation REAL,
        PrecipitationType VARCHAR(20),
        Conditions VARCHAR(100),
        Description TEXT

    );
"""

```

```

# Create table
cur.execute("DROP TABLE IF EXISTS project;")
cur.execute(create_table)

for _, row in final_df.iterrows():
    values = (
        row['month/year'], row['state'], row['covid_cases'], row['fips_code'],
        row['civilian_population'],
        row['labor_force'], row['population_percent'], row['employment'],
        row['employment_percent'],
        row['unemployment'], row['unemployment_percent'],
        row['employment_to_population_ratio'], row['unemployment_rate'],
        row['TempMax'], row['Temp'], row['TempMin'], row['FeelsLike'], row['Humidity'],
        row['Precipitation'],
        row['PrecipitationType'], row['Conditions'], row['Description']
    )
    cur.execute("""
        INSERT INTO project (
            month_year, state, covid_cases, fips_code, civilian_population, labor_force,
            population_percent,
            employment, employment_percent, unemployment, unemployment_percent,
            employment_to_population_ratio,
            unemployment_rate, TempMax, Temp, TempMin, FeelsLike, Humidity,
            Precipitation, PrecipitationType,
            Conditions, Description
        ) VALUES (
            %s, %s, %s, %s, %s, %s, %s,
            %s, %s, %s, %s, %s,
            %s, %s, %s, %s, %s, %s, %s,
            %s, %s, %s
        );
    """, values)

# query table
conn.commit()

cur.execute("SELECT * FROM project LIMIT 5;")
rows = cur.fetchall()

print("First 5 rows:")
for row in rows:
    print(row)

# close connection
cur.close()

```

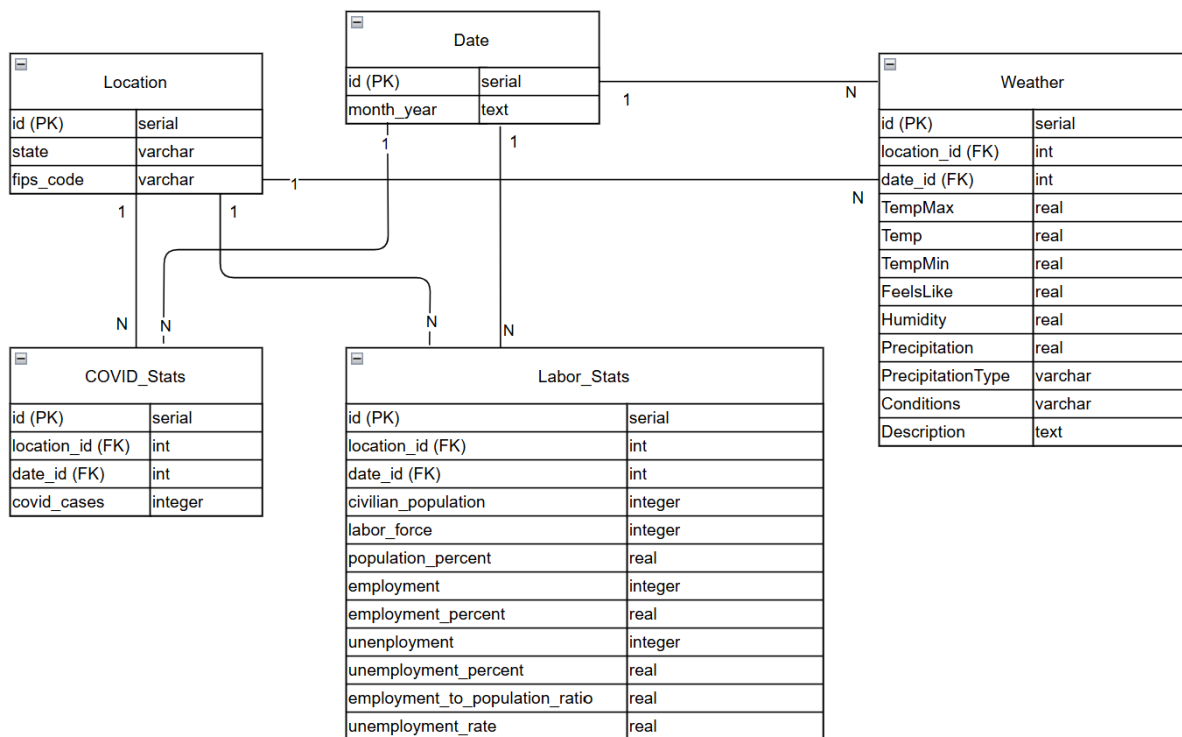
```
conn.close()
```

```
# %%
```

Database Schema

- The schema has:
 - 3 fact tables: covid data, unemployment data, weather data.
 - 2-dimension tables: Date (month/year), Location (state, FIPS) which are essentially reference tables for month/year and state name.

Merges between the 3 datasets were done by the month/year column and state column in each cleaned up dataset. The location and date tables are the 2-dimension tables for the columns that the 3 fact tables have in common. The schema is set up to allow one-to-many relationships between the Date table (month/year column) and the month/year column in the weather, covid, and labor tables. It also allows one-to-many relationships between the Location table (state column) and the state column in the weather, covid, and labor tables. Refer to *erd_diagram.pdf* for relationships and data types below:



Automation

To automate the data pipeline, we used Apache Airflow to orchestrate the transformation and integration of the three cleaned datasets: COVID-19 case counts, unemployment statistics, and weather data. The Python script (`clean_data_sql_merge.py`) from the data transformation process was created to standardize column names, merge the datasets by state and month/year, load data to the Postgres database, and generate a single unified dataset called `combined_data.csv`. This script was integrated into an Airflow DAG (`etl_pipeline_dag`) to ensure repeatability and automation. The DAG was configured to run manually from the Airflow UI and provides a reliable mechanism to regenerate the merged dataset whenever updated inputs are available.

`Etl_pipeline_dag.py`:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import subprocess

default_args = {
    'owner': 'airflow',
    'start_date': datetime(2024, 1, 1),
    'retries': 1
}

dag = DAG(
    dag_id='etl_pipeline_dag',
    default_args=default_args,
    schedule_interval=None,
    catchup=False,
    description='Run clean_data_merge_sql.py to load data, clean it, and load into db'
)

def run_clean_script():
    subprocess.run(["python3", "/home/jhu/python_scripts/clean_data_sql_merge.py"],
check=True)

merge_task = PythonOperator(
    task_id='merge_cleaned_data',
    python_callable=run_clean_script,
    dag=dag
)
```

API

To enable access to the merged dataset, a RESTful API was developed using Flask. The API reads from `combined_data.csv` and offers several endpoints to query and retrieve aggregated information. Users can access summaries such as total COVID-19 cases, average unemployment

rates, and average temperature by state. Additionally, a filter endpoint allows users to narrow results by specific state and/or month. The API returns all responses in JSON format and is designed to be lightweight, accessible, and easy to integrate with external dashboards or analytical tools.

An `api_request.py` file was created to allow users to call to the API and return the data in a JSON format. Users can customize this if they would like to filter to specific states/months/years.

`Api_app.py`:

```
from flask import Flask, jsonify, request
import pandas as pd
from pathlib import Path

app = Flask(__name__)

data_path = Path("/home/jhu/data/combined_data.csv")

# load in csv
df = pd.read_csv(data_path)

@app.route("/")
def home():
    return {"message": "Welcome to the Weather, COVID, and Unemployment API"}

@app.route("/cases-by-state", methods=["GET"])
def cases_by_state():
    result = df.groupby("state")["covid_cases"].sum().reset_index()
    return jsonify(result.to_dict(orient="records"))

@app.route("/avg-unemployment-by-state", methods=["GET"])
def avg_unemployment_by_state():
    result = df.groupby("state")["unemployment_rate"].mean().reset_index()
    return jsonify(result.to_dict(orient="records"))

@app.route("/avg-temp-by-state", methods=["GET"])
def avg_temp_by_state():
    result = df.groupby("state")["Temp"].mean().reset_index()
    return jsonify(result.to_dict(orient="records"))

@app.route("/filter", methods=["GET"])
def filter_data():
    state = request.args.get("state")
    month = request.args.get("month/year")
    filtered_df = df.copy()

    if state:
        filtered_df = filtered_df[filtered_df["state"].str.lower() == state.lower()]
```

```

    if month:
        filtered_df = filtered_df[filtered_df["month/year"] == month]

    return jsonify(filtered_df.to_dict(orient="records"))

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

api_request.py:

```

import requests

base_url = "http://localhost:5000"

# output is home route
response = requests.get(f"{base_url}/")
print("Home:", response.json())

# output is average cases by state
response = requests.get(f"{base_url}/cases-by-state")
print("Cases by state:", response.json())

# output is average unemployment by state
response = requests.get(f"{base_url}/avg-unemployment-by-state")
print("Avg unemployment:", response.json())

# avg temperature by state route
response = requests.get(f"{base_url}/avg-temp-by-state")
print("Avg temperature:", response.json())

# filter state
params = {
    "state": "New York",
    "month": "1/1/2021"
}
response = requests.get(f"{base_url}/filter", params=params)
print("Filtered data:", response.json())

```

Instructions to Run the Pipeline

1. Extract zip folder into your "...\\jhu_docker\\shared" folder
2. Open PowerShell or command prompt
3. Navigate using `cd ...\\jhu_docker\\shared\\final_project_container\\final_project`
4. Run "docker-compose up"

Output is:


```
PS C:\Users\tsosa\Documents\shared\jhu_docker_new\jhu_docker\shared\final_project_container\final_project> docker-compose up
time="2025-04-21T20:02:32-04:00" level=warning msg="C:\Users\tsosa\Documents\shared\jhu_docker_new\jhu_docker\shared\final_project_co
ntainer\final_project\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential conf
usion"
[+] Running 4/0
✔ Container final_project-mongodb-1          Running                                0.0s
✔ Container final_project-postgres-1         Running                                0.0s
✔ Container final_project-airflow-jupyter-1  Created                                 0.0s
✔ Container final_project-flask-api-1        Created                                 0.0s
Attaching to airflow-jupyter-1, flask-api-1, mongodb-1, postgres-1
flask-api-1 | 'FLASK_ENV' is deprecated and will not be used in Flask 2.3. Use 'FLASK_DEBUG' instead.
flask-api-1 | * Serving Flask app 'api_app'
```

5. Type “V” to view in docker, ensure containers are running:

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last start...	Actions		
☐	<u>final_project</u>	-	-	-	0.96%	16 minutes ago			
☐	airflow-jupyter-1	b721871e0735	final_project-airflow-j	8080:8080 Show all ports (2)	0%	16 minutes ago			
☐	postgres-1	98ee3f439726	postgres:16.3	5432:5432	0%	16 minutes ago			
☐	mongodb-1	be3757220e70	mongo:7.0.10-rc0-jan	27018:27018	0.96%	16 minutes ago			

6. **Open new terminal**, then **cd** into:

“...\\jhu docker\\shared\\final project container\\final project\\airflow”

7. Run:

```
docker exec -it final_project-airflow-jupyter-1 bash
```

```
PS C:\Users\tsosa\Documents\shared\jhu_docker_new\jhu_docker\shared\final_project_container\final_project\airflow> docker exec -it final_project-airflow-jupyter-1 bash
root@cd4e233fbbf31:/home/jhu#
```

8. Then, run:

```
airflow webserver --port 8080
```

```
PS C:\Users\tsosa\Documents\shared\jhu_docker_new\jhu_docker\shared\final_project_container\final_project\airflow> docker exec -it final_project-airflow-jupyter-1 bash
root@cd4e233fbf31:/home/jhu# airflow webserver --port 8080

-----
---- /| |---()-----//-----//-----//| |//
--//---|////////---//---//|//|//
Running the Gunicorn Server with:
Workers: 4 sync
Host: 0.0.0.0:8080
Timeout: 120
Logfiles: - -
Access Logformat:
=====
/usr/local/lib/python3.10/dist-packages/flask_limiter/extension.py:324 UserWarning: Using the in-memory storage for tracking rate limits as no storage was explicitly specified. This is not recommended for production use. See: https://flask-li
```

9. Open another terminal and navigate to airflow folder (final project/airflow), run:

docker exec -it final_project-airflow-jupyter-1 bash

10. Run:

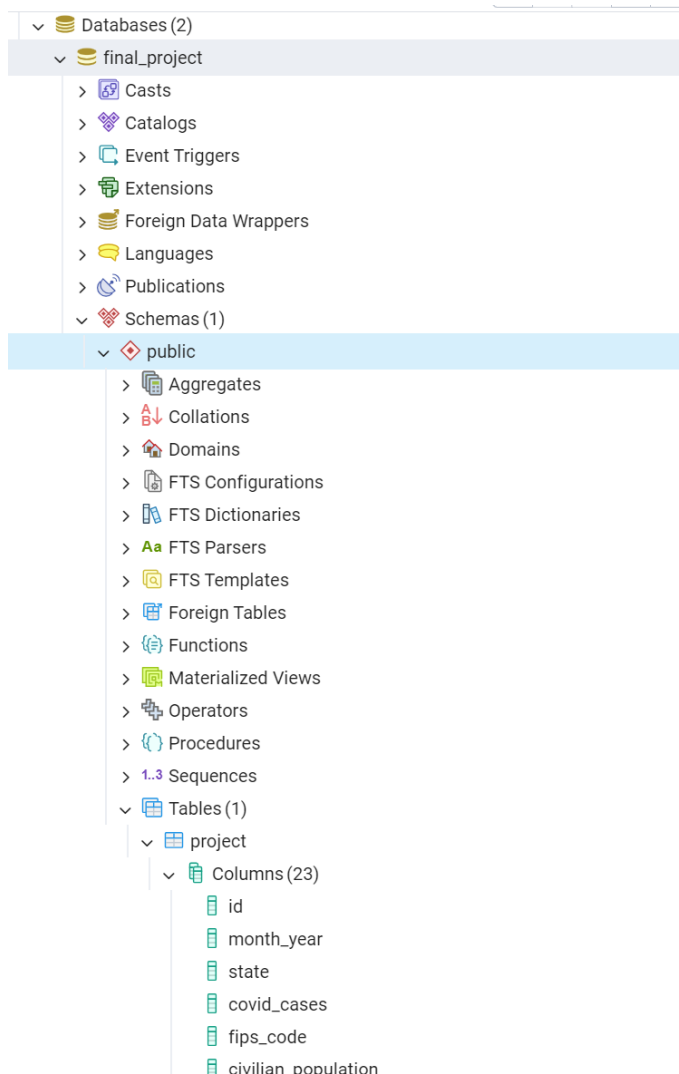
airflow scheduler

- » DAG Run
etl_pipeline_dag / ▶ 2025-04-19, 20:51:38 UTC

△ Details **Graph** 📅 Gantt <> Code

merge_cleaned_data
■ success
PythonOperator

12. (Optional) Confirm data is loaded into PostgreSQL database:



13. Open new terminal, navigate to api folder (final_project/api), run:

`docker exec -it final_project-airflow-jupyter-1 python3 /home/jhu/api/api_app.py`

```
PS C:\Users\tsosa\Documents\shared\jhu_docker_new\jhu_docker\shared\final_project_container\final_project\api> docker exec -it final_project-airflow-jupyter-1 python3 /home/jhu/api/api_app.py
* Serving Flask app 'api_app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.21.0.5:5000
Press CTRL+C to quit
```

14. In browser, go to <http://localhost:5000/>, output is:

15. Open new terminal, navigate to api folder (final_project/api), run:

Output is json data from API request:

16. (Optional) If you'd like to filter to specific states/months or request less data, customizing can be done in the API folder in the `api request.py` file.